The Job Sequencing and Tool Switching Problem

Part II: The Non-Compact Configuration-Based Formulation

Scope. This document presents the *non-compact, configuration-based* ILP formulation for the Uniform SSP, proposed by Colares and Wagler [Colares and Wagler(2026)]. It includes: the configuration space and job clusters, the transition metric with a metric-space proof, the full ILP with PCTSP-based subtour elimination (which is tighter than GSECs), and the column generation framework with the H-SUKP pricing subproblem. This is the primary modelling framework for all subsequent analysis.

Key caveat: the PCTSP-based formulation requires *both* column generation (exponential variables) and row generation (exponential constraints), creating a "double generation" problem. Part IV (`04_mtz_formulation.tex`) resolves this by replacing the exponential constraints with polynomial MTZ constraints, leaving only column generation.

Contents

Motivation: Escaping Compact-Model Failure

As established in `01_foundations.tex`, all compact ILP formulations for the SSP suffer from two fundamental problems: (i) the LP relaxation collapses to a lower bound of 0, providing no pruning information, and (ii) the 0-block symmetry creates 2^k equivalent integer solutions that exhaust the branch-and-bound tree.

The non-compact formulation introduced here replaces positional variables with explicit *configuration* variables — one per valid magazine state. This eliminates the symmetry problem and gives a much tighter LP relaxation. However, it introduces a new computational challenge that must be understood upfront.

The double-generation problem. The non-compact formulation has *two* sources of exponential size simultaneously:

- **Exponential columns** (configurations): $|\mathcal{C}| = \binom{m}{b}$ variables. Handled by **column generation**: add improving configurations one at a time via a pricing oracle (H-SUKP, Section 5.3).
- **Exponential rows** (subtour constraints): the PCTSP-based subtour elimination constraints number $O(2^{|\mathcal{C}|})$. Handled by **row generation**: add violated constraints on-the-fly via separation.

Running column generation and row generation *simultaneously* is called *double generation*. It is technically complex: each LP re-solve after adding a new column potentially violates previously satisfied subtour cuts, requiring a full re-separation pass. This interaction makes convergence slow and implementation fragile.

The remedy (Part IV, `04_mtz_formulation.tex`): replace the exponential subtour constraints with a *polynomial* set of Miller–Tucker–Zemlin (MTZ) constraints. This eliminates row generation entirely, leaving only column generation — a much more tractable problem. The trade-off is a slightly weaker LP bound, but in practice this is far outweighed by the reduction in implementation complexity and solver iterations.

The non-compact approach takes a different viewpoint from compact models: instead of asking "which tool is at position k?", it asks "which complete magazine state (configuration) is used to process a group of jobs?" By working directly over the set of all valid magazine states, the model eliminates positional symmetry entirely and achieves a far tighter LP relaxation.

Configuration Space and Job Clusters

We use the same notation as `01_foundations.tex`: $J = \{1, \ldots, n\}$ jobs, $T = \{1, \ldots, m\}$ tools, magazine capacity b, requirement sets $T_j \subseteq T$.

Definition 2.1 (Configuration space $\mathcal{C}$)**.** The *configuration space* is the set of all maximal tool subsets:

$$\mathcal{C} = \{C \subseteq T \mid |C| = b\}.$$

Its cardinality is $|\mathcal{C}| = \binom{m}{b}$.

By Lemma 1 of `01_foundations.tex`, restricting to maximal configurations is without loss of generality. We augment $\mathcal{C}$ with a dummy configuration $C_0 = \emptyset$ (or any fixed initial state) representing the machine's magazine before any job is processed.

Definition 2.2 (Job cluster H_j)**.** For each job $j \in J$, its *cluster* is the set of configurations that can process it:

$$H_j = \{C \in \mathcal{C} \mid T_j \subseteq C\}.$$

Since $|T_j| \leq b$, every cluster is non-empty.

Key point. Unlike in a standard GTSP, clusters can *overlap*: a single configuration C may satisfy multiple jobs simultaneously when $T_i \cup T_j \subseteq C$. This overlap is the central structural property of the SSP and is handled by the GTSP equivalence in `03_gtsp_equivalence.tex`.

The Transition Metric

Definition

When the machine transitions from a magazine state C_u to C_v, the tools in $C_u \setminus C_v$ are evicted and the tools in $C_v \setminus C_u$ must be inserted. Since both configurations are maximal ($|C_u| = |C_v| = b$), the number of evictions equals the number of insertions, and the switching cost is:

$$d(C_u, C_v) = |C_v \setminus C_u| = b - |C_u \cap C_v|.$$

Metric Space Validity

Theorem 3.1 (Triangle inequality for d [Colares and Wagler(2026)])**.** *For any three configurations $C_u, C_w, C_v \in \mathcal{C}$:*

$$d(C_u, C_v) \leq d(C_u, C_w) + d(C_w, C_v).$$

Proof. Using $d(A, B) = |A \setminus B|$, we show $|C_u \setminus C_v| \leq |C_u \setminus C_w| + |C_w \setminus C_v|$.
Pick any tool $t \in C_u \setminus C_v$, so $t \in C_u$ and $t \notin C_v$.

- If $t \notin C_w$: then $t \in C_u \setminus C_w$.
- If $t \in C_w$: then since $t \notin C_v$, we have $t \in C_w \setminus C_v$.

In both cases $t \in (C_u \setminus C_w) \cup (C_w \setminus C_v)$, so $C_u \setminus C_v \subseteq (C_u \setminus C_w) \cup (C_w \setminus C_v)$. Taking cardinalities and applying subadditivity of set union:

$$|C_u \setminus C_v| \leq |(C_u \setminus C_w) \cup (C_w \setminus C_v)| \leq |C_u \setminus C_w| + |C_w \setminus C_v|. \qquad \square$$

$\square$

Remark 3.2. The metric is also symmetric ($d(C_u, C_v) = d(C_v, C_u)$) and non-negative, so $(\mathcal{C}, d)$ is a valid metric space. Symmetry follows from $|A \setminus B| = |B \setminus A|$ when $|A| = |B| = b$. The maximum distance is b (completely disjoint configurations).

The Non-Compact ILP

Decision Variables

- $y(v) \in \{0, 1\}$: 1 if configuration $v \in \mathcal{C}$ is *activated* (used at least once in the sequence).
- $x(u, v) \in \{0, 1\}$: 1 if the machine transitions *directly* from configuration u to configuration v.

We include C_0 (dummy start/end node) with $y(C_0) = 1$ fixed and $d(C_0, v)$ equal to the cost of loading configuration v from an empty magazine.

Objective and Base Constraints

$$\min \quad \sum_{u \in \mathcal{C}} \sum_{v \in \mathcal{C} \setminus \{u\}} d(u,v)\, x(u,v) \tag{1}$$

$$\text{s.t.} \quad \sum_{v \in H_j} y(v) \geq 1 \quad \forall j \in J \tag{2}$$

$$\sum_{u \neq v} x(u,v) = y(v) \quad \forall v \in \mathcal{C} \tag{3}$$

$$\sum_{v \neq u} x(u,v) = y(u) \quad \forall u \in \mathcal{C} \tag{4}$$

$$y(C_0) = 1 \tag{5}$$

$$x(u,v) \in \{0,1\},\ y(v) \in \{0,1\} \quad \forall u, v \in \mathcal{C}$$

Interpretation. Constraint (2) ensures every job is processed by at least one activated configuration. Constraints (3)–(4) enforce flow conservation: each activated node has exactly one incoming and one outgoing arc, forming a collection of paths/cycles. The depot constraint (5) anchors the sequence to start and end at C_0.

Subtour Elimination: PCTSP-Based SECs

Constraints (3)–(4) alone allow disconnected subtours among activated configurations. We need constraints that force all activated configurations to lie on a *single* path through the depot.

The standard approach uses Generalized Subtour Elimination Constraints (GSECs). However, GSECs are only valid when $y(v) \geq 1$ for all nodes in the subset, which does not hold here since $y(v) \in \{0,1\}$. A tighter and correct formulation uses the Prize-Collecting TSP (PCTSP) subtour elimination constraints [Balas(1989)]:

$$\sum_{u \in S} \sum_{v \in S \setminus \{u\}} x(u,v) \ \leq \ \sum_{u \in S \setminus \{k\}} y(u) \quad \forall S \subset \mathcal{C},\ 2 \leq |S| \leq |\mathcal{C}| - 1,\ C_0 \notin S,\ \forall k \in S.$$

Remark 4.1 (Why PCTSP over GSEC)**.** The GSEC form $\sum_{u,v \in S} x(u,v) \leq |S| - 1$ assumes all $y(u) = 1$ within S. When some $y(u) = 0$ (unactivated), the bound is too loose and subtours of inactive nodes can appear in LP solutions. The PCTSP form $\sum x(u,v) \leq \sum_{u \neq k} y(u)$ correctly adapts to the actual number of *activated* nodes, eliminating spurious subtours even in the fractional LP relaxation. See [Balas(1989)] for the original derivation.

Complete Non-Compact ILP

Combining all components:

$$
\begin{aligned}
\min \quad & \sum_{u,v} d(u,v)\, x(u,v) && \\
\text{s.t.} \quad & \sum_{v \in H_j} y(v) \geq 1 && \forall j \in J \\
& \sum_{u \neq v} x(u,v) = y(v) && \forall v \in \mathcal{C} \\
& \sum_{v \neq u} x(u,v) = y(u) && \forall u \in \mathcal{C} \\
& \sum_{u \in S} \sum_{v \in S \setminus \{u\}} x(u,v) \leq \sum_{u \in S \setminus \{k\}} y(u) && \forall S \subset \mathcal{C},\, C_0 \notin S,\, \forall k \in S \\
& y(C_0) = 1 && \\
& x(u,v),\, y(v) \in \{0,1\} && \forall u, v \in \mathcal{C}
\end{aligned}
$$

Since $|\mathcal{C}| = \binom{m}{b}$ is exponential, the full model cannot be instantiated explicitly. This motivates the column generation approach in the next section.

Column Generation Framework

Why generate configurations, not whole sequences. A tempting alternative is a Dantzig–Wolfe master over entire job sequences σ: one column per sequence with cost $\mathrm{KTNS}(\sigma)$ and convexity $\sum_\sigma \lambda_\sigma = 1$. This degenerates. Every sequence processes every job exactly once, so each job-covering row $\sum_\sigma a_{j\sigma}\lambda_\sigma = 1$ is identical to the convexity row; the LP collapses to $\min_\sigma \mathrm{KTNS}(\sigma) = Z^*_{\mathrm{SSP}}$, giving no usable lower bound, and its pricing subproblem is itself an NP-hard SSP. Generating *configurations* (equivalently, arcs of the configuration graph), as below, instead yields a genuine LP relaxation whose pricing is a tractable set-union knapsack. *(This observation was relocated here from the gap-analysis note, Part V, where it did not belong.)*

Restricted Master Problem (RMP)

Let $V \subset \mathcal{C}$ be a small, dynamically growing subset of configurations (initially seeded by heuristic solutions or greedy construction). The RMP solves the LP relaxation of the non-compact ILP restricted to V:

$$
\begin{aligned}
\min \quad & \sum_{i,j \in V} d(i,j)\, x_{ij} && && (6) \\
\text{s.t.} \quad & \sum_{i \in V \cap H_r} y_i \geq 1 && \forall r \in J \quad (\text{dual: } \pi_r \geq 0) && (7) \\
& \sum_{j \neq i} x_{ij} = y_i && \forall i \in V \quad (\text{dual: } \alpha_i \in \mathbb{R}) && (8) \\
& \sum_{j \neq i} x_{ji} = y_i && \forall i \in V \quad (\text{dual: } \beta_i \in \mathbb{R}) && (9) \\
& \sum_{i \in S} \sum_{j \in S \setminus \{i\}} x_{ij} \leq \sum_{i \in S \setminus \{k\}} y_i && \forall S \subset V,\, C_0 \notin S,\, k \in S \quad (\text{dual: } \mu_{S,k} \leq 0) && (10) \\
& y_{C_0} = 1 && && \\
& x_{ij} \geq 0,\; y_i \geq 0 && \forall i, j \in V &&
\end{aligned}
$$

At each iteration, solving the RMP yields LP optimal value Z^*_{RMP} and dual variables $(\pi, \alpha, \beta, \mu)$.

Pricing Subproblem: When to Add a Configuration

A new configuration $c \notin V$ should be added if doing so can reduce the RMP objective. The *reduced cost* of introducing c with incoming arc from $u \in V$ and outgoing arc to $w \in V$ is:

$$\bar{c}(u, c, w) = d(u, c) + d(c, w) - \alpha_u - \beta_w - \sum_{r \in J:\, T_r \subseteq c} \pi_r - \sum_{S,k:\, u,c \in H(S)} \mu_{S,k}$$

(the GSEC dual terms vanish for a new node c not yet in any identified subset S). A configuration c is *improving* if $\min_{u,w} \bar{c}(u, c, w) < 0$.

Ignoring routing duals for the pricing step (standard practice), the dominant term is the coverage gain. We seek:

$$\max_{c:\, |c| = b} \sum_{r \in J:\, T_r \subseteq c} \pi_r$$

subject to $|c| \leq b$ and the configuration feasibility condition $T_r \subseteq c \iff v_r = 1$ (all tools of job r are in c).

The Pricing Subproblem as a Hypergraph SUKP

Let $z_t \in \{0, 1\}$ indicate that tool t is included in the new configuration, and $v_r \in \{0, 1\}$ indicate that job r is *covered* by the configuration (meaning all its required tools are selected). The pricing problem is:

$$\max \quad \sum_{r \in J} \pi_r \, v_r \tag{11}$$

$$\text{s.t.} \quad v_r \leq z_t \qquad \forall r \in J,\, \forall t \in T_r \tag{12}$$

$$\sum_{t \in T} z_t \leq b \tag{13}$$

$$z_t \in \{0, 1\} \;\; \forall t \in T, \quad v_r \in \{0, 1\} \;\; \forall r \in J$$

Constraint (12) ensures that $v_r = 1$ only if all tools in T_r are selected. This is the *Set Union Knapsack Problem (SUKP)* with hyperedges (one hyperedge per job, connecting its required tools) and item capacity b: it is **strongly NP-hard** [Goldschmidt et al.(1994)Goldschmidt, Nehme, and Yu]. It can be solved exactly by branch-and-bound or IP, with ML-based acceleration for large instances (see `08_research_notes.md`).

Remark 5.1 (SUKP vs. standard knapsack)**.** In a standard knapsack, each item contributes independently to value and weight. In the SUKP, a job r contributes reward π_r only if *all* its tools are selected simultaneously — a non-separable, conjunctive structure. This is why the pricing subproblem is strictly harder than a standard knapsack.

Column Generation Termination

The LP relaxation is solved to optimality when no improving column can be found: $\min_c \bar{c}(u, c, w) \geq 0$ for all potential insertion pairs (u, w). This certificate is provided by the exact SUKP solver. If a negative reduced cost column is found, it is added to V and the RMP is re-solved. The process continues until optimality is certified. Integer solutions are recovered by branch-and-price [Barnhart et al.(1998)Barnhart, Johnson, Nemhauser, Savelsbergh, and Vance].

Why the Non-Compact Formulation is Better

1. **No positional symmetry.** Each configuration is a unique physical machine state, not a positional variable. The 2^k 0-block symmetry of compact models disappears: the KTNS policy is implicit in the choice of which configurations appear on the path.

2. **Tight LP relaxation.** The coverage constraints over job clusters, combined with flow conservation, provide a much stronger lower bound than the zero bound of position-based models.

3. **Scalable via column generation.** We never enumerate all $\binom{m}{b}$ configurations; only those with improving reduced costs are added.

4. **Natural GTSP structure.** The non-compact ILP is isomorphic to a Generalised TSP over the configuration graph (Part III), enabling the use of decades of TSP theory and solvers.

5. **The double-generation caveat.** The PCTSP-based formulation above requires both column generation and row generation simultaneously. Part IV (`04_mtz_formulation.tex`) resolves this by replacing the exponential PCTSP constraints with $O(n^2)$ MTZ constraints, eliminating row generation entirely at the cost of a slightly weaker LP bound.

References

[Balas(1989)] E. Balas. The prize collecting traveling salesman problem. *Networks*, 19(6):621–636, 1989.

[Barnhart et al.(1998)Barnhart, Johnson, Nemhauser, Savelsbergh, and Vance] C. Barnhart, E. L. Johnson, G. L. Nemhauser, M. W. P. Savelsbergh, and P. H. Vance. Branch-and-price: Column generation for solving huge integer programs. *Operations Research*, 46(3):316–329, 1998.

[Colares and Wagler(2026)] R. Colares and A. Wagler. Exact formulations for the job sequencing and tool switching problem. *Working Paper, Université Clermont Auvergne, LIMOS*, 2026.

[Goldschmidt et al.(1994)Goldschmidt, Nehme, and Yu] O. Goldschmidt, D. Nehme, and G. Yu. Note on the set-union knapsack problem. *Naval Research Logistics*, 41(6):833–842, 1994.